ORIENTATION — read this first

The entry point to ABRA. What the project is, the two principles that govern it, what each model does, and the failure mode it keeps hitting.

Drafted by Cowork; every figure measured and filled by Claude Code against the store at commit `8cd1593` *on 2026-07-25. Where a number here disagrees with an older document, this one was measured and the older one was not.*

What this project is

ABRA ingests public Pokémon Showdown replays for Champions VGC 2026 Reg M-B, models the metagame, and produces `data/meta-usage.json`.

CHOMP answers one question at team preview: which four to bring, which two to lead. It reads ABRA's model. When ABRA improves, CHOMP improves without a code change.

portfolio presents both. Evidence first, marketing never.

The whole thing exists to answer a decision problem, not to produce statistics. Every model earns its place by improving a decision or by honestly failing to.

The one principle that governs the data

Store raw, analyse on top.

Every replay is archived whole in `data/games.ladder.raw-logs.jsonl`. The parsed store (`data/games.ladder.jsonl`) is a *derived* view. A new question is a **re-parse** (`MODE=reparse`), never a re-pull. Changing how games are segmented is a **re-filter**.

Never design an analysis that requires re-fetching replays.

This has already paid for itself twice. Measuring the format's true paralysis rate needed the `|cant|` lines, which the parsed store discards — the raw archive still had them. And on 2026-07-25 a reparse recovered **356 games** that the parsed store had lost but the archive had kept.

The archive must stay complete or this principle is a lie. The hourly GitHub Action appends to the store, but `raw-logs.jsonl` is gitignored, so a CI-ingested game gets no local raw log — 453 games were in that state. Two guards now exist:

- `MODE=backfill` refetches raw logs for any stored game missing from the archive.
- `MODE=reparse` **refuses to run** while any stored game lacks a raw log, because reparse replaces the store with whatever the archive can rebuild and would delete the rest.

The second principle: know which games a number came from

`data/quality-filter.json` is the single definition of a usable game. `engine/quality.js` and `engine/quality.py` are thin readers of it — neither hard-codes a threshold, and `tests/test-quality.js` asserts both select an identical set of ids.

<!-- BEGIN:FUNNEL --> Of **88,091** games collected, **22,905** are usable — **26.0%**.

Games are dropped for five reasons, in this order:

Stage	Games remaining
collected	88,091
after removing named bots	49,866
after removing accounts that behave like bots	44,089
after removing forfeits	28,778
after removing games under 3 turns	28,556
after requiring all four brought to be revealed	**22,905**

<!-- END:FUNNEL -->

1. **Named bots** — accounts whose usernames announce them.
2. **Behavioural bots** — accounts that play like a script regardless of name. The decisive signal is **team invariance**: an account playing hundreds of games without ever changing a slot.
3. **Forfeits** — a forfeit records who quit, not who was winning.
4. **Games under three turns** — not enough play to reveal a bring or a lead trade.
5. **Partial brings** — `brought` is what the replay REVEALED, not what was selected.

Two limitations that must be stated whenever these games are used:

- **Bot detection is a floor, not a proof.** Call the surviving set "no bot detected", never "human".
- **Requiring a full bring conditions on game length.** A bring statistic computed here is "the bring, among games long enough to show it" — which is not "the bring".

Why the bot filter is not paranoia

A small number of undetected bot accounts played the **same six Pokémon** across a large share of the store. Those six were exactly the species whose usage collapsed once the accounts were removed, and the pre-filter top of the usage table *was* that team.

The model CHOMP reads was, for a period, a description of one script's team rather than of the metagame. That is the concrete reason this filter exists and why every engine must go through it.

<!-- BEGIN:RAWREADERS --> **28 engine tools still read the store with neither the clean filter nor a declared reason.** `engine/selftest.js` fails while any remain, and names them:

`engine/argmax_paired.js` , `engine/bench_speed_consolidate.js` , `engine/calibrate.py` , `engine/click_census.js` , `engine/click_counts.js` , `engine/coach.js` , `engine/derive_sets.js` , `engine/durable-ingest.js` , `engine/feature_engine_contrast.js` , `engine/forced_switch_audit.js` , `engine/joint_click_census.js` , `engine/medicham2-browser.js` , `engine/mega_census.js` , `engine/mega_sets_from_sheets.js` , `engine/mew_farm.js` , `engine/next_regulation_ingest.js` , `engine/replay_differential.js` , `engine/rollout_r1_join.py` , `engine/rollout_switch_census.js` , `engine/sheet_usage.js` , `engine/smogon_coverage.js` , `engine/stamp.js` , `engine/validate_store.js` , `tests/test-medicham-coverage.js` , `tests/test-next-regulation.js` , `tests/test-parse.js` , `tests/test-side-guard-chooser.js` , `tests/test-workflow-paths.js`

Anything they publish is computed over a store that is 74.0% unusable. <!-- END:RAWREADERS -->

The models

Engine	Question it answers
`analyze.js`	What does the metagame look like? → `meta-usage.json` **, which CHOMP reads**

Engine	Question it answers
`guru.py`	Which archetypes beat which?
`roles.py`	What jobs can each species do? (multi-label, credibility-gated, **52 roles**)
`nmf_roles.py`	What roles emerge from the data without being named in advance?
`war.py`	How much is a species worth, controlling for teammates? (ridge RAPM)
`xatu*.py`	What is the opponent's set, given what has been revealed?
`pory.py`	What is the live win probability?
`slowking*.py`	What is the equilibrium at preview?
`chomp_ev.js`	Does bring quality beat a coin?
`medicham2-browser.js`	The hand-written doubles rollout engine
`champions_sim.js`	The **official** Showdown Champions simulator
`illusion.js`	Which Zoroark were disguised? (by legality contradiction)
`sanity_check.py`	**96** assertions across the whole system

The engineering standards, and why each exists

Every standard in `docs/ARCHITECTURE.md` was written after a specific failure. They are not preferences.

- **S1 — single source of truth.** Where several representations are unavoidable, one is definitive and the rest are **generated by a script**, never hand-synchronised.
- **S2 — duplication that cannot be removed must be observable.** A consumer-driven contract test asserts every implementation agrees. This caught engine drift on its first run.
- **S7 — the store has a shape, and it is tested.** No duplicate ids; `brought ⊆ six`; `lead ⊆ brought`; the winner is one of the two players.
- **S8 — measured, never asserted.** No constant that affects a result is typed by hand.
- **S9 — golden master before refactoring.** Record the outputs, change the code, compare.
- **S10 — enumerate closed domains.** 25 natures, 18 types, 6 stat stages: walk the whole domain and assert direction. A spot-check cannot detect a missing row, because a missing row usually degrades to a plausible default rather than an error.
- **S11 — one publisher.** Exactly one process commits and pushes.

The failure mode this project keeps hitting

Silent wrongness. Not crashes — the code returns a number, and the number is wrong.

Every serious defect found so far shares a shape:

- A table that was **incomplete** rather than incorrect, so counting it looked fine. *(The nature table held 23 of 25; a missing nature falls through to the neutral multiplier.)*
- A check **aimed away from where faults occur.** *(The duplicate scan read the first 5,000 lines of an append-only log, where duplicates enter at the end.)*
- A **plausible causal story** that nobody verified, which then propagated into several documents. *(`merge -X ours` was blamed for the store duplication in four documents. The actual cause was `merge=union` in `.gitattributes`, which concatenates both sides of a conflicting hunk — and which applies to `rebase` as well as `merge`, so switching to rebase did not stop it.)*
- A number **retracted in the changelog** but left standing elsewhere. *(The role-pair median cell of `n = 7,971` was retracted in 2.7.0; the real figure is ~52 across 1,137 cells. It still appears in the white paper, `ROLE-FAMILY.md` and `PUBLICATION.md`.)*

The countermeasure is not care. It is: assert direction rather than count, aim checks at the region where the fault occurs, treat causal claims as hypotheses until tested, and make a retraction fail a build rather than rely on someone remembering.

Where the rules actually come from

Champions is a real Showdown format with its own mod. `data/mods/champions/` in the Showdown **master** branch (not the npm package) defines the stat system, the status mechanics, the mega formes and the damage scripts.

That mod is the authority. Where our engine and the mod disagree, the mod is right. See `docs/ADR-001-use-the-champions-mod.md` — the decision is to stop maintaining a parallel rules implementation and drive the official simulator instead.

Stat convention: Champions uses **SP**, not EVs. `stat = base + SP + 20`; `HP = base + SP + 75`.

Who does what

	Cowork	Claude Code
Prose, design, review, literature	yes	—
Measuring anything	**never**	yes
Running engines, tests, the simulator	**never**	yes
git, pushing	**never**	yes
Deciding whether a draft is correct	—	yes

Cowork writes only to `docs/_inbox/`. Claude Code writes only to `docs/_outbox/`. Single writer per folder, so the two cannot collide.

Cowork never authors a number. Any figure that is not `<<MEASURED>>` in a Cowork draft should be treated as suspect and verified independently.

How to read the rest of the docs

Document	Purpose
`ARCHITECTURE.md`	The standards and the faults that produced them
`ADR-001-use-the-champions-mod.md`	Why the rules engine is being replaced
`MODELS.md`	Per-model living ledger
`ABRA-whitepaper.md`	Technical, with maths and cited sources
`ABRA-deck-plain-english.md`	Plain English; links the white paper
`ABRA-technical-docs.md`	ASD-STE100, organised by Diátaxis
`DEFENSE.md`	Statistical defense of each design decision: the formal result, the citation, and the measurement that confirms or contradicts it
`METHODOLOGY.md`	Why the experiments are designed the way they are, with the literature and the measurements that confirm or contradict it
`BACKLOG.md`	What is known-available and unused, what is left, and what the project is ultimately for
`CHANGELOG.md`	Newest first; the top version matches the artefacts

A caution for anyone reading the older docs: several carry figures that were later retracted or superseded, and the causal claim about the store duplication is wrong in the older text. Reconciling those is active work. **Where a document disagrees with a measurement, the measurement wins.**

Honest limitations

- **The store spans a short window.** No temporal design is possible; any claim about meta drift is unsupported.
- **The clean subset is small** — 1,061 games. Several results will not clear zero on it. That is the finding, not a reason to loosen the filter.
- **Sets are mostly unknown.** Across 72,367 observed sets, a mean of **1.38 of four moves** is revealed, **69.7%** have no item, and **75.5%** have no ability. Whatever fills those gaps **dominates** any simulation result — this invalidated two engine comparisons before it was caught.
- **The hand-written rollout engine disagrees materially with the official simulator** — 31.1 points of win probability on average, flipping the favourite in 3 of 8 matchups. Until ADR-001 lands, treat every rollout-derived number as provisional.